

1. Server.java — o ponto de entrada

```
“public static void main(String[] args) throws IOException {  
    HttpServer server = HttpServer.create(new InetSocketAddress(8080), 0);  
    server.createContext("/students", new StudentHandler());  
    server.setExecutor(null);  
    server.start();  
}”
```

- Cria um servidor HTTP escutando na porta 8080.
- `createContext("/students", new StudentHandler())` é a linha mais importante:
Ela diz que toda requisição cujo caminho comece com `/students` deve ser tratada pelo `StudentHandler`. Isso vale tanto para `/students` quanto para `/students/1`, `/students/42`, etc — o `HttpServer` faz esse roteamento por prefixo automaticamente.
- `setExecutor(null)` faz o servidor usar uma thread padrão simples (não é multithread eficiente, mas serve para exemplo/estudo).
- `server.start()` inicia o servidor, que fica ouvindo indefinidamente.

2. StudentHandler.java — 0

"controller"

```
switch (exchange.getRequestMethod()) {  
    case "GET": handleGet(exchange); break;  
    case "POST": handlePost(exchange); break;  
    case "PUT": handlePut(exchange); break;  
    case "DELETE": handleDelete(exchange); break;  
    default: sendResponse(exchange, 405, ...);  
}
```

- Implementa a interface **HttpHandler**
 - **HttpServer** chama o método **handle(HttpExchange exchange)** para qualquer requisição em **/students***
- Somente analisa o método e delega para um método especializado
 - **GET /students** → lista tudo
 - **POST /students** → cria um novo
 - **PUT /students/1** → atualiza o id 1
 - **DELETE /students/1** → apaga o id 1

2.1 handleGet — buscar dados

- GET /students:

```
"if (path.equals("/students")) {  
    String json = JsonUtil  
        .getMapper().  
        writeValueAsString(repository.findAll());  
    sendResponse(exchange, 200, json);  
    return;  
}"
```

- GET /students/{id}:

```
"String[] parts = path.split("/");  
if (parts.length != 3) { sendResponse(exchange, 400, ...); return; }  
int id = Integer.parseInt(parts[2]);  
Student student = repository.findById(id);"
```

2.2 handlePost — criar um novo estudante

```
“String body = new String(exchange.getRequestBody().readAllBytes());  
Student student = JsonUtil.getMapper().readValue(body, Student.class);  
Student createdStudent = repository.create(student);”
```

- Lê o corpo bruto da requisição (`exchange.getRequestBody()`).
- Usa o Jackson (`JsonUtil.getMapper().readValue`) para converter esse texto JSON em um objeto Java `Student`.
- Passa esse objeto para `repository.create(student)`, que efetivamente salva.
- Serializa o estudante criado (agora com id) de volta para JSON e responde com status 201 Created

2.3 handlePut e handleDelete

Seguem o mesmo padrão de `handleGet` (id): extraem o id da URL, e:

- `handlePut`: lê o corpo (novos dados), chama `repository.update(id, ...)`, retorna 200 ou 404.
- `handleDelete`: chama `repository.delete(id)`, e se der certo responde 204 No Content (sem corpo), senão 404.

2.5 sendResponse — método utilitário compartilhado

```
“private void sendResponse(HttpExchange exchange, int statusCode, String response) throws IOException {  
    exchange.getResponseHeaders().set("Content-Type", "application/json");  
    exchange.sendResponseHeaders(statusCode, response.getBytes().length);  
    try (OutputStream output = exchange.getResponseBody()) {  
        output.write(response.getBytes());  
    }  
}”
```

Todos os outros métodos (**handleGet**, **handlePost**, etc.) chamam esse no final.

Ele centraliza a lógica de "montar a resposta HTTP":

- Define o header **Content-Type: application/json**
- Escreve o código de status e o tamanho do corpo
- Escreve o JSON no stream de saída.

Evita repetir esse código em cada handler.

3. StudentRepository.java — a "camada de dados" (em memória)

```
“private final List<Student> students = new ArrayList<>();  
private Integer nextId = 1;”
```

- Construtor: já popula a lista com dois estudantes de exemplo (Lucas e Emma), chamando **create()** internamente.
- **findAll()**: retorna a lista inteira.
- **findById(id)**: percorre a lista comparando id
- **create(student)**: atribui um id sequencial (**nextId++**), adiciona na lista, retorna o objeto criado.
- **update(id, updatedStudent)**: encontra o estudante pelo id e sobrescreve os campos (nome, email, matrícula, curso) — mantendo o mesmo id.
- **delete(id)**: encontra e remove da lista, retorna **true/false** conforme sucesso.

4. JsonUtil.java — utilitário de serialização

```
“public final class JsonUtil {  
    private static final ObjectMapper MAPPER = new ObjectMapper();  
    public static ObjectMapper getMapper() { return MAPPER; }  
}”
```

Uma classe simples com um único **ObjectMapper** (do Jackson) estático e compartilhado.

Em vez de criar um **new ObjectMapper()** toda vez (que é uma operação com algum custo), a aplicação inteira reutiliza essa única instância para converter objetos Java ↔ JSON.

Resumo do fluxo completo (exemplo: POST)

- Postman envia POST `http://localhost:8080/students` com corpo JSON.
- `Server` recebe a conexão na porta 8080 e, como o path começa com `/students`, delega ao `StudentHandler`.
- `StudentHandler.handle()` vê que o método é `"POST"` e chama `handlePost(exchange)`.
- `handlePost` lê o corpo, usa `JsonUtil` para desserializar em `Student`, chama `StudentRepository.create()`.
- `StudentRepository` gera um novo id e guarda o estudante na lista em memória.
- `handlePost` serializa o resultado de volta para JSON via `JsonUtil` e chama `sendResponse`.
- `sendResponse` escreve os headers, status 201 e o corpo JSON de volta — que o Postman recebe como resposta